

IT 106 Final Project

Integrative Programming and Technologies

Final Course Project

PROJECT TITLE: Development of an Integrated Web-Based Application Using Modern Programming Technologies

Group Size: 1-4 (maximum of 4 is subject for justification) members per group

Project Type: Web-based Application

Required Output: Working system, documentation, database file, presentation, and source code

Presentation: Final project defense/demo schedule to be announced

Late Submission: Subject to point deduction or may be marked incomplete depending on the reason and submission delay

Submission Platform: LMS

Deadline of Submission for Non-Graduating Students: May 25, 2026 @ exactly 12:00NN

Deadline of Submission for Graduating Students: May 20, 2026 @ exactly 12:00NN

I. Project Overview

For the final project in **IT 106 – Integrative Programming and Technologies**, students are required to design, develop, and present a functional web-based application that demonstrates the integration of multiple programming concepts, technologies, and software development practices discussed in the course.

The project must apply key IT 106 concepts such as:

- Web application development
- RESTful API integration
- Database connectivity
- Data mapping and exchange
- JSON/XML handling
- Middleware or backend services
- Frontend and backend integration
- Object-oriented programming concepts
- Design patterns
- Interfaces
- Inheritance
- Basic security and validation
- Testing and documentation

The goal of this final project is to assess the students' ability to create a working system that integrates different technologies into one complete application.

II. General Project Requirement

Each group must develop a **web-based CRUD application** that allows users to create, read, update, delete, search, and manage records using a connected database.

The application must have:

1. **Frontend interface**
 2. **Backend/server-side logic**
 3. **Database integration**
 4. **REST API endpoints**
 5. **Authentication or user validation**
 6. **Data exchange using JSON**
 7. **Proper project documentation**
 8. **Final presentation and demonstration**
-

III. Suggested Project Titles

Students may choose from the following or propose their own project title:

1. **Student Information Management System**
 2. **Online Appointment Scheduling System**
 3. **Event Registration and Attendance Monitoring System**
 4. **Inventory Management System**
 5. **Library Book Borrowing System**
 6. **Clinic Patient Record Management System**
 7. **Faculty Workload Management System**
 8. **Room Reservation System**
 9. **Task and Project Monitoring System**
 10. **Online Enrollment Pre-Registration System**
 11. **Lost and Found Management System**
 12. **Equipment Borrowing and Tracking System**
 13. **Scholarship Application Monitoring System**
 14. **Capstone/Thesis Proposal Tracking System**
 15. **Internship Application and Monitoring System**
-

IV. Required Technologies

Students may use any of the following technology stacks:

Option A: JavaScript-Based Stack

- Frontend: HTML, CSS, JavaScript, Vue.js, React, or plain JavaScript
- Backend: Node.js with Express.js
- Database: PostgreSQL, MySQL, MongoDB, or SQLite
- API Format: JSON

Option B: PHP-Based Stack

- Frontend: HTML, CSS, JavaScript, Bootstrap, Tailwind CSS, or Vue.js
- Backend: PHP or Laravel
- Database: MySQL, PostgreSQL, or SQLite
- API Format: JSON

Option C: Hybrid Stack

- Frontend: Vue.js, React, or plain JavaScript
 - Backend: Laravel, Node.js, Express.js, or another approved backend framework
 - Database: MySQL, PostgreSQL, MongoDB, or SQLite
 - API Format: JSON
-

V. Minimum System Features

Each final project must include the following features:

1. User Interface

The system must have a clear and user-friendly interface with:

- Homepage or dashboard
- Navigation menu
- Form pages
- Data table/list view
- Search/filter section
- Buttons for add, edit, delete, and view
- Basic responsive layout

2. CRUD Functionality

The system must support complete CRUD operations:

Function	Description
<i>Create</i>	Add new records
<i>Read</i>	Display records from the database
<i>Update</i>	Edit existing records
<i>Delete</i>	Remove records from the database

Example records may include students, products, appointments, books, events, or transactions.

3. REST API Implementation

The system must include API endpoints such as:

Method	Endpoint Example	Purpose
GET	/api/students	Retrieve all records
GET	/api/students/:id	Retrieve one record
POST	/api/students	Add new record
PUT/PATCH	/api/students/:id	Update record
DELETE	/api/students/:id	Delete record

The API must return data in **JSON format**.

4. Database Integration

The system must connect to a database and include:

- At least **3 related database tables**
- Primary keys and foreign keys
- Proper data types
- Basic constraints
- Sample records
- Database schema or ERD

Example:

Table	Purpose
users	Stores user accounts
students	Stores student information
transactions	Stores system activity records

5. Authentication or User Validation

The project must include at least one of the following:

- Login and logout
- User role validation
- Password validation
- Form validation
- Session or token-based authentication

Minimum requirement: the system must validate user input before saving data to the database.

6. Data Mapping and Exchange

The project must demonstrate data exchange using JSON.

Example:

```
{
  "id": 1,
  "name": "Juan Dela Cruz",
  "course": "BSIT",
  "year_level": "3rd Year"
}
```

Students may also include XML, CSV, or JSON import/export as an additional feature.

7. Object-Oriented Programming Requirement

The project must demonstrate at least one of the following:

- Class and object implementation
- Inheritance
- Interface
- Abstract class
- Service class
- Controller class
- Repository class
- Model class

Example:

```
class User {
  constructor(id, name, email) {
    this.id = id;
    this.name = name;
    this.email = email;
  }
}

class Student extends User {
  constructor(id, name, email, course) {
    super(id, name, email);
    this.course = course;
  }
}
```

8. Design Pattern Requirement

The project must apply at least **one design pattern**, such as:

Design Pattern	Possible Use
MVC	Separating model, view, and controller
Singleton	Database connection manager
Factory Method	Creating user or record objects
Façade	Simplifying complex service calls
Repository Pattern	Managing database queries
Observer	Notification or event listener
Decorator	Adding extra behavior to an object

Minimum requirement: students must explain where the design pattern is applied in their project.

VI. Project Documentation Format

Each group must submit a project documentation file with the following parts:

1. Title Page

Include:

- Project title
- Course title: IT 106 – Integrative Programming and Technologies
- Section
- Group number
- Names of members
- Instructor's name
- Date submitted

2. Introduction

Briefly describe the background, purpose, and importance of the proposed system.

3. Objectives of the Project

Include:

General Objective

To design and develop an integrated web-based application that demonstrates CRUD operations, database connectivity, API integration, and application of programming concepts.

Specific Objectives

The system aims to:

- 1. Develop a functional web-based application.
- 2. Implement CRUD operations.
- 3. Integrate a database for record management.
- 4. Create REST API endpoints for data exchange.
- 5. Apply object-oriented programming concepts.
- 6. Use at least one software design pattern.
- 7. Test the system based on functional requirements.

4. Scope and Limitations

Describe what the system can and cannot do.

Example:

The system focuses on managing student records, including adding, updating, deleting, searching, and viewing student information. However, it does not include online payment processing or integration with external university systems.

5. System Features

List and describe the major features of the system.

Example:

Feature	Description
Login	Allows authorized users to access the system
Dashboard	Displays system summary
Add Record	Allows users to create new records
Manage Records	Allows users to view, edit, delete, and search records
Reports	Displays basic summaries or printable data

6. System Architecture

Include a simple architecture diagram or explanation.

Example:



7. Database Design

Include:

- Entity Relationship Diagram
- List of tables
- Table fields
- Primary keys
- Foreign keys
- Sample data

Example:

Field Name	Data Type	Description
id	integer	Primary key
name	varchar	Name of record
email	varchar	Email address
created_at	timestamp	Date created

8. API Documentation

Include the API routes used in the system.

Method	Endpoint	Description
GET	/api/items	Get all items
POST	/api/items	Add new item
PUT	/api/items/:id	Update item
DELETE	/api/items/:id	Delete item

9. Screenshots of the System

Include screenshots of:

- Login page
- Dashboard
- Add form
- Data table
- Edit form
- Delete confirmation
- Search/filter result
- API test using Postman, Thunder Client, or browser

10. Testing Results

Use this table format:

Test Case	Expected Output	Actual Output	Status
Add new record	Record is saved	Record was saved	Passed
Edit record	Record is updated	Record was updated	Passed
Delete record	Record is removed	Record was removed	Passed
Search record	Matching data appears	Matching data appeared	Passed

11. Conclusion

Summarize what the group developed and what concepts were applied.

12. References

Include references for frameworks, libraries, tutorials, and documentation used.

VII. Final Presentation Requirements

Each group will present and demonstrate their system.

The presentation must include:

1. Project title and group members
2. Problem or background
3. System objectives
4. Technologies used
5. System architecture
6. Database design
7. API endpoints
8. System demonstration
9. Testing results
10. Challenges encountered
11. Conclusion

Recommended presentation duration: **10–15 minutes per group.**

VIII. Required Submission Files

Each group must submit the following:

1. **Project documentation** in PDF or Word format
2. **Source code folder**
3. **Database file or SQL dump**

4. **Presentation file**
5. **Screenshots of system output**
6. **Short user manual**
7. **GitHub repository link, if available**
8. **Screenshots of API testing using Postman, Thunder Client, or browser**

Suggested file naming format: *IT106_FinalProject_GroupNumber_ProjectTitle*

Example: *IT106_FinalProject_Group03_StudentInformationSystem*

IX. Final Project Rubric

Total: 100 Points

Criteria	Description	Points
System Functionality	Completeness of features, CRUD operations, working pages, and correct outputs	25
Backend and API Integration	Proper API routes, request handling, JSON responses, and server-side logic	15
Database Design and Integration	Proper database structure, relationships, constraints, and successful database connection	15
Frontend Design and Usability	User-friendly interface, layout, responsiveness, navigation, and form design	10
Application of OOP and Design Patterns	Use of class, inheritance/interface/abstract class, MVC or other design pattern	10
Validation and Basic Security	Input validation, authentication, error handling, and safe data handling	10
Testing and Debugging	Test cases, screenshots, and evidence that the system was tested	5
Documentation	Completeness, clarity, formatting, diagrams, and explanation of system components	5
Presentation and Demonstration	Clarity of presentation, actual system demo, and ability to answer questions	5
Total		100

X. Individual Contribution Rating

Each member must submit an individual contribution report.

Member Name	Assigned Task	Output Completed	Contribution Rating
Member 1	Frontend development	Completed UI pages	25%
Member 2	Backend/API	Completed API routes	25%
Member 3	Database design	Completed schema and SQL file	25%
Member 4	Documentation/testing	Completed documentation and test cases	25%

XI. Suggested Grading Interpretation

Score Range	Interpretation
90–100	Excellent
85–89	Very Good
80–84	Good
75–79	Satisfactory
Below 75	Needs Improvement

XII. Final Instruction to Students

This final project serves as the culminating requirement for IT 106. Students are required to design, develop, document, and present a functional web-based application that demonstrates the integration of frontend, backend, database, REST API, data exchange, object-oriented programming, and selected design pattern concepts discussed in the course.

The system must be original, functional, properly documented, and presented during the scheduled final project demonstration. During the demonstration, each member must be able to explain the specific part of the system that he/she contributed.

A project with complete documentation but without a **working system demonstration** cannot receive full credit for system functionality.

XIII. Student Submission Checklist

Before uploading, each group must ensure that they have submitted the following:

Requirement	Status
Final project documentation in Word/PDF format	☐

Source code folder or GitHub repository link	☐
Database SQL dump or database setup file	☐
Presentation slides	☐
Screenshots of the system output	☐
Screenshots of API testing using Postman, Thunder Client, or browser	☐
Short user manual	☐
Individual contribution report	☐

XIV. Academic Integrity Reminder

All final project outputs must be **original** or properly credited. Any copied system, reused project, or downloaded source code submitted without proper modification and explanation may result in point deductions, project revision, or an incomplete grade.

Students must be able to explain their own code, database structure, API endpoints, and system workflow during the final demonstration.

End

Prepared by:

JOHN MICHAEL F. CUTAMORA
Faculty, Computer Science Department
College of Computing and Information Sciences